Next steps in the organisation's Azure sandbox

The POC was built and measured on synthetic data in an independent subscription. This page describes how it moves into the organisation's environment and how accuracy is measured on the organisation's own historical documents. Nothing here is new design: it is the same Terraform, code and scripts, pointed at a different tenant.

1. Stand up the environment

- **Subscription and quota.** A sandbox subscription with a resource group and a storage account for Terraform state. Quota requests that gated the POC and should be filed first: Azure OpenAI Data Zone capacity for `gpt-5.2` (learn model), `gpt-5.4-mini` (delta, classification, chat) and `text-embedding-3-large`; Content Understanding; Azure SQL in the application region; AI Search.
  – **The gpt-5.2 deployment is also subject to the subscription's own risk check** (ADR 0004 amendment 1). The POC's subscription is still refused by its account risk check (last tried 2026-10-26), so every measured figure in the pack is on gpt-5.4-mini. Set `deploy_gpt52 = false` and `learn_model = "gpt-5.4-mini"` in `sandbox.tfvars` until a gpt-5.2 deployment succeeds on the organisation's subscription. Then run the learn-model study (corpus v5 on gpt-5.2 against the gpt-5.4-mini run; the runbook's next-wake items) and switch only if gpt-5.2 meets ADR 0004 amendment 1's rule.
- **Terraform.** Copy `infra/environments/sandbox.tfvars.example` to `sandbox.tfvars` and fill its placeholders: the owner tag, the SQL admin group (name and object id), the identity team's tenant and API audience, the cost-export start date and the alert distribution list. Review the plan with the cloud architect, then apply. Model deployments wait on quota. While the `gpt-5.2` quota is pending, set `deploy_gpt52 = false`: the `gpt-5.2` deployment is skipped and learning uses `gpt-5.4-mini` (app setting, model version and Content Understanding alias), with no other change.
- **Posture.** Any environment other than the POC refuses HTTP Basic sign-in and turns on diagnostic settings to Log Analytics and SQL auditing. It scopes identities to single secrets and containers, and leaves the Defender for AI plan and resource-provider registration to the subscription owner (`docs/decisions/0027-sandbox-posture-and-subscription-level-terraform.md`, `infra/README.md`). Private endpoints are a follow-up.
- **Identity.** An Entra app registration with the six app roles (Applicant, Underwriter, UW Manager, Admin, Engineer, QuoteSystem), mapped to groups. Either the identity team creates it and hands over the tenant and audience, or Terraform creates it (`create_entra_app = true`). HTTP Basic sign-in is off and cannot be turned on outside the POC.
- **What the quote system supplies: the quote.**
  – The quote id is the quote-management system's key, and the quote exists there before any upload.
  – Outside the POC, `Quotes__ApplicantMayCreate` is false (set by Terraform from `environment`): an Applicant upload to an unknown quote is refused with 404.
  – Until the quote system's integration creates quotes, an underwriter does (ADR 0024).
- **Schema.** `lossrun-db deploy` applies the rerunnable Liquibase-style changesets; `lossrun-db audit-proof` must pass all four checks before anything else runs.
- **Smoke test.** Generate the synthetic corpus (`lossrun eval generate --seed 8`), run it through the deployed API, score it. The numbers must match the POC's within their confidence intervals; that proves the environment, not the product.
- **Pipeline.** Every pull request runs the build and the tests (.NET, web, end-to-end). Every change to `infra/` also runs `terraform fmt`, `validate` and the offline posture tests (`.github/workflows/terraform.yml`, no sign-in). A signed-in `terraform plan` against the sandbox is added once the pipeline has its service connection.
- **SQL for the tests without Docker.** A SQL server (or elastic pool) set aside for tests in the sandbox is the test target, never the one holding the working database: set `LOSSRUN_TEST_SQL` to it, with a login that may

create databases; the tests create, use and drop their own `lossrun_tests_*` databases (`src/README.md`, *SQL for the tests*). LocalDB 2022 on a developer's machine works too.

2. Real documents: the tuning set

- **Truth export.** The data team exports one CSV from the claims system with `carrier, policy_number, claim_number, date_of_injury, valuation_date, indemnity_paid, indemnity_reserve, medical_paid, medical_reserve, expense_paid, total_incurred`. This is the ground truth; no labelling is needed.
- **Document pool.** About 400–600 loss runs from the archive (a 100-document tuning set plus a 300–500-document blind set; `docs/operations/blind-run.md` §1) across all carriers and input types (native PDF, scan, fax, spreadsheet, bundles), copied into a folder. `lossrun eval import` builds the manifest with a hash and input type per file.
- **Seal.** An evaluator who is not the author runs `lossrun eval seal` with a published seed: about 100 documents released as the tuning set, 300–500 sealed as the blind set, file hashes published so the set cannot change.
- **Tune.** The tuning set runs through the pipeline: carrier formats are learned once, rule thresholds and glossary gaps are adjusted. Results on this set are never reported as accuracy. `lossrun tune` runs it in batches with a ranked fix list and a stopping rule; `docs/operations/playbook.md` is the assistant-driven version of the whole path.

3. The blind run: the number that counts

The procedure and the scoring rules every entrant is measured by are written down in the repository and travel with the code.

- The learned-format registry is reset the day before, so the build starts from the same position as any vendor.
- `lossrun eval run --mode api` submits the sealed set to the deployed pipeline (about an hour and about $45–60 for 500 documents, derived from synthetic measurements and replaced after the tuning run; `docs/operations/blind-run.md`).
- `lossrun eval align` matches each extracted claim to the keyed record; amounts count as truth only when valuation dates agree, everything else goes to adjudication, blind to the entrant.
- `lossrun eval score` produces per-field, per-carrier and per-input-type accuracy with confidence intervals, review rate, model calls and cost per document, stamped with the code version and model versions.
- The same `score` command runs on each vendor's output in the canonical JSON schema, so all entrants are compared on identical documents under identical rules.

Every synthetic number in the acceptance table is replaced by this run, not added to.

4. After the blind run

- A shadow period alongside the manual process, tracking re-keying minutes, quote cycle time and accuracy on live submissions, before any cut-over.
- Backfill of the historical archive if chat and analytics over history are wanted: the same pipeline as a batch job, mostly zero-cost replays by then.

Decisions that belong to the organisation

- Thresholds for the acceptance criteria (the requirement leaves them as "XX"); proposed values accompany the scoring protocol.
- Retention period for the locked immutability policies (7 years proposed).

- Whether the chat service gets a dedicated model deployment with Prompt Shields before or after the shadow period.
- Length of the shadow period and the metrics it is judged on.